


```

        cur = cur->next;
    }
}
}
cout << "原始链表: ";
printList(head);
Solution s;
ListNode* reversedHead = s.reverseList(head);
cout << "反转后的链表: ";
printList(reversedHead);

// 释放链表内存
while (reversedHead) {
    ListNode* temp = reversedHead;
    reversedHead = reversedHead->next;
    delete temp;
}
}
return 0;
}

```

2、运行结果

```

3
1 2 3 4 5
原始链表: 1 2 3 4 5
反转后的链表: 5 4 3 2 1
1 2
原始链表: 1 2
反转后的链表: 2 1

原始链表:
反转后的链表:

Process returned 0 (0x0)    execution time : 10.785 s
Press any key to continue.

```

3、实习总结与体会

不仅学习到了反转链表，还学习了如何像操作输入流（如cin）那样操作字符串。

（1）反转链表：`cur` 指针初始化为链表的头节点用于遍历链表；`pre` 指针初始化为空指针用于记录当前节点的前一个节点，反转后它将成为新链表的头节点。当所有节点都被反转后，`pre` 指向新的头节点，即原链表的最后一个节点。返回 `pre` 作为反转后的新链表的头。

（2）按行读入输入数据：因为有 `t` 个测试用例，所以需要按行读入来分割多个测试用例。`cin.ignore()` 用于忽略输入缓冲区中的换行符，确保后续的 `getline` 能够正确读取整行。`istringstream s(line)` 用于将字符串流将行分割成多个整数。`while (s >> value)` 用于逐个读取整数值。

（3）释放内存：遍历反转后的链表并释放每个节点的内存，防止内存泄漏。